

SCOMP Reference Sheet

Processes

- **pid_t fork();**
 - creates a new process by duplicating the calling process
- **unsigned int sleep(unsigned int seconds);**
 - sleep for a specified number of seconds
- **pid_t getpid(void);**
- **pid_t getppid(void);**
 - get process identification
- **pid_t wait(int* wstatus);**
- **pid_t waitpid(pid_t pid, int* wstatus, int options);**
 - wait for a process to terminate
- **void exit(int status);**
 - cause normal process termination

Exec functions

- **int execlp(const char *file, const char *arg, ...);**
- **int execvp(const char *file, char *const argv[]);**
 - replaces the current process image with a new process image

Signals

- **int kill(pid_t pid, int sig);**
 - send signal identified by *sig* to a process identified by *pid*
- **int pause(void);**
 - wait for signal
- **int raise(int sig);**
 - send a signal identified by *sig* to the calling process
- **unsigned int alarm(unsigned int seconds);**
 - set an alarm clock for delivery of a signal to the calling process
- **int sigaction(int signum, const struct sigaction* act, struct sigaction* oldact);**
 - examine and change a signal action
- **int sigprocmask(int how, const sigset_t* set, sigset_t* oldset);**
 - examine and change blocked signals
- **int sigpending(sigset_t* set);**
 - examine pending signals
- **int sigismember(const sigset_t* set, int signo);**
 - test for a signal in a signal set

Pipes

- **int pipe(int pipefd[2]);**
 - *pipefd* is used to return two file descriptors referring to the ends of the pipe
- **size_t read(int fd, void* buf, size_t count);**
 - read from a file descriptor
- **size_t write(int fd, const void* buf, size_t count);**
 - write to a file descriptor
- **int close(int fd);**
 - close a file descriptor
- **int dup(int oldfd);**
- **int dup2(int oldfd, int newfd);**
 - duplicate a file descriptor

Shared memory

- **int shm_open(const char* name, int oflag, mode_t mode);**
 - creates and opens a new, or opens an existing, shared memory object
- **int shm_unlink(const char* name);**
 - removes a shared memory object
- **int ftruncate(int fd, off_t length)**
 - truncate a file to a specified length
- **void *mmap(void* addr, size_t length, int prot, int flags, int fd, off_t offset);**
 - maps a shared memory object into the process' address space
- **int munmap(void* addr, size_t len);**
 - disconnects the shared memory area from the process' address space

Semaphores

- **sem_t *sem_open(const char* name, int oflag);**
- **sem_t *sem_open(const char* name, int oflag, mode_t mode, unsigned int value);**
 - creates and opens a new, or opens an existing, semaphore
- **int sem_close(sem_t* sem);**
 - close a semaphore
- **int sem_wait(sem_t* sem);**
 - decrements (locks) the semaphore pointed to by *sem*
- **int sem_post(sem_t* sem);**
 - increments (unlocks) the semaphore pointed to by *sem*
- **int sem_unlink(const char* name);**
 - removes the semaphore referred to by *name*

Threads

- **int pthread_create(pthread_t* thread, pthread_attr_t* attr, void *(*start_routine) (void *), void* arg);**
 - starts a new thread in the calling process
- **int pthread_join(pthread_t thread, void **retval);**
 - waits for the thread specified by *thread* to terminate
- **void pthread_exit(void *retval);**
 - terminates the calling thread and returns a value via *retval*
- **pthread_t pthread_self(void);**
 - obtain ID of the calling thread
- **int pthread_mutex_init(pthread_mutex_t* mutex, pthread_mutexattr_t* attr);**
 - initializes a mutex in the calling process
- **int pthread_mutex_destroy(pthread_mutex_t* mutex);**
 - destroy the mutex object referenced by *mutex*
- **int pthread_mutex_lock(pthread_mutex_t* mutex);**
- **int pthread_mutex_trylock(pthread_mutex_t* mutex);**
 - locks the mutex object pointed to by *mutex*
- **int pthread_mutex_unlock(pthread_mutex_t* mutex);**
 - unlocks the mutex object referenced by *mutex*
- **int pthread_cond_init(pthread_cond_t* cond, pthread_condattr_t* attr);**
 - initializes a condition variable in the calling process
- **int pthread_cond_destroy(pthread_cond_t* cond);**
 - destroy the condition variable referenced by *cond*
- **int pthread_cond_broadcast(pthread_cond_t* cond);**
- **int pthread_cond_signal(pthread_cond_t* cond);**
 - broadcast or signal a condition referenced by *cond*
- **int pthread_cond_wait(pthread_cond_t* cond, pthread_mutex_t* mutex);**
- **int pthread_cond_timedwait(pthread_cond_t* cond, pthread_mutex_t* mutex, struct timespec* abstime);**
 - wait on a condition referenced by *cond*